

CHAPTER 01

Eval Engineering

The discipline of building production-grade evaluation systems that govern AI behavior at scale.

"You can't improve what you can't measure. You can't measure what you haven't defined."

MENTAL MODEL SHIFT

OLD MODEL	EVAL ENGINEERING
Evals are debugging tools	→ Evals are governance systems
Run once before deployment	→ Run continuously in production
Generic accuracy metrics	→ Domain-specific precision
Sample 10% of traffic	→ Cover 100% of traffic
Static test sets	→ Adaptive evaluation
Cost center	→ Strategic asset
Measures behavior	→ Governs behavior

WITHOUT EVALS

The Endless Pilot Loop

Demos well, never graduates to production.

The Firefighting Trap

Reacting to failures instead of preventing them.

The Rebuild Tax

Every project reconstructs infra from scratch.

The Accuracy Plateau

Stuck at 70%, nothing moves the needle.

5-STAGE LIFECYCLE

- 01

🔗

LLM-as-Judge

60-70%

Write a prompt defining 'good'. Fast to set up but 70% means wrong 3/10 times.
- 02

👥

SME Refinement

90-95%

Domain experts review failures, identify patterns, articulate criteria.
- 03

🔄

Develop with Evals

CI/CD

Benchmark and A/B test. Regression testing in CI/CD.
- 04

🔧

Scale with SLMs

<150MS

Fine-tune 3-8B models. Same accuracy at 10-40× lower cost.
- 05

🛡️

Deploy Guardrails

100%

Real-time scores powering governance. Block risky outputs.

EVAL ENGINEERING LIFECYCLE



WITH VS WITHOUT

CHORE	WITHOUT EVALS	WITH EVALS
Model upgrade	Weeks of manual testing	Run suite overnight, ship in days
Prompt change	Hope nothing broke	Know exactly what changed
Bug report	Guess and check	Reproduce, fix, add to regression
New hire	Tribal knowledge	Read the test cases

LLM-as-Judge

<> CODE-BASED EVALS

Exact match, regex, schema
~\$0 <10ms 100% accurate
Always start here first.

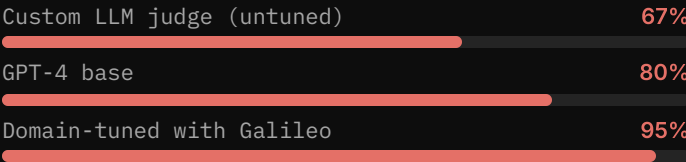
☙ LLM JUDGE EVALS

Semantic quality, open-ended
\$0.01-0.10 2-4s 90-95%
Use where judgment is subjective.

7 Principles for Judge Prompts

- 1 **Be specific.** "Is it helpful?" is untestable. "Does it include a step-by-step solution?" is. Define exactly what pass and fail look like.
- 2 **Clarify vague terms.** Replace "appropriate tone" with "formal, no slang, addresses user by name." If two people could disagree on the rating, rewrite it.
- 3 **Binary output.** Pass or Fail only. Likert scales just move the ambiguity from the answer to the score.
- 4 **Few-shot examples.** Show the judge what good and bad look like. Include edge cases, the ones right on the boundary.
- 5 **Explicit decision rules.** Remove wiggle room. "If no source is cited, mark FAIL regardless of accuracy." Hard rules prevent soft reasoning.
- 6 **One judge, one criterion.** "Helpful but factually wrong" breaks a single judge. Split it: one judges helpfulness, another judges accuracy.
- 7 **Reasoning before verdict.** Make the judge think before ruling. Chain-of-thought before the final label boosts accuracy.

ACCURACY PROGRESSION



ELEMENT	BASIC	REFINED
Role	None	Clear domain context
Criteria	Vague	Numbered, testable
Examples	None	Few-shot + reasoning
Output	Ambiguous	Reasoning then pass/fail
Accuracy	~60%	75-80%

CHAINPOLL

Multi-Judge Voting

0.781

AUROC

Single LLM judge = noisy (66-68%). Fix: 5 judges voting in parallel. Same prompt to 5× gpt-3.5-turbo. Each gives Pass/Fail + CoT. Score = (#Fail)/5.

Example: 3 Fail, 2 Pass = 0.60 hallucination risk

METHOD	AUROC	VS CP
ChainPoll (5)	0.781	Baseline
SelfCheck-Bert	0.673	-14%
SelfCheck-NGram	0.644	-18%
G-Eval	0.579	-26%
GPTScore	0.524	-33%

\$0.015/eval vs \$0.060 GPT-4. 4× cheaper.

3 Biases That Kill Accuracy

⬆️ Position Bias

35% of judgments change based on answer order.

FIX Swap positions, run pairwise twice.

☰ Verbosity Bias

LLMs prefer longer answers 85% of the time.

FIX 'Evaluate quality independently of length.'

🗑️ Self-Preference Bias

Models favor own outputs by 10-25%.

FIX Different model for judge vs generator.

CHAINPOLL SETTINGS

TEMPERATURE 0	JUDGES 5 parallel	SCORE No. of pass / 5
------------------	----------------------	--------------------------

F1 SCORE BENCHMARKS

≥0.95 Outstanding	≥0.85 Excellent	0.70-0.85 Substantial	<0.50 Poor
----------------------	--------------------	--------------------------	---------------

CHAPTER 03

SME Refinement

CRITIQUE SHADOWING – 7 STEPS

1👑

Expert Defines Quality

One 'benevolent dictator'

2📦

Build Dataset

Features × scenarios × personas

3✎

Write Critiques

Pass/Fail + reasoning

4🗑

Fix Broken Systems

Don't eval what's broken

5👥

Build Judge

Critiques as few-shots

6📊

Track Metrics

F1, Kappa, ICC

7🔄

Iterate

>90% agreement

Ground Truth Dataset

- ✓ 200-500 representative examples

✓ Label with documented reasoning

✓ Split: Train 20% / Dev 40% / Test 40%
- ✓ Include known failure modes

✓ Cohen's Kappa* > 0.8

✓ Refresh quarterly
- ★ **Cohen's Kappa** – measures how well two raters agree on categorical labels, correcting for chance. Ranges from -1 (complete disagreement) to 1 (perfect agreement); 0 = no better than random.

MIN VIABLE GROUND TRUTH

100 examples by lead expert with documented reasoning. At least 20 per failure mode.

DO

- ✓ 5-10 examples
- ✓ Balance pass/fail
- ✓ Diverse scenarios
- ✓ Borderline cases
- ✓ Match prod distribution
- ✓ Clear critiques

DON'T

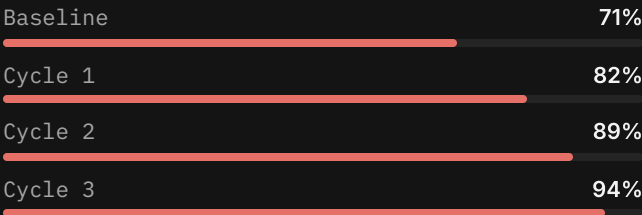
- ✗ All passing examples
- ✗ All similar scenarios
- ✗ Restating verdicts
- ✗ Synthetic without SME check
- ✗ Skewing to edge cases only
- ✗ No held-out test set

CASE STUDY: BANKING CHATBOT

About: Chatbot should help user with banking related queries and never give investment advice.

71% → 94%

The metric in this example tracks that the finance chatbot should not give financial advice as it's against the laws.



DISCOVERIES

Cycle 1: Educational vs Advice

+11%

Educational explanations ≠ advice.

Cycle 2: Sentiment as Advice

+7%

'Strong momentum' counts as advice.

Cycle 3: The Gray Zone

+5%

'Should I invest in X?' needs deflection.

TIME SPENT

Labeling 150 examples	4h
Disagreement review 3×30	5h
Prompt iteration	2h
Docs + handoff	1h
Total	12h

CHAPTER 04

Scaling with SLMs

PROBLEM \$50k/day, 2-4s latency, 1% sampling

FIX 10-40× cheaper, 15-150ms, 100% coverage

Why LLM Judges Break at Scale

Cost Ceiling

\$10k-50k/day for M conversations. Multiply per metric. Sampling misses clustered failures.

Latency Barrier

1-3s per eval in the critical path. With 5 evals per output, sequential = 15s. Parallel execution bounds it to ~3s, but still too slow for real-time guardrails.

Prompting Ceiling

Last 5-6% resists prompting. Prompt grows, attention fragments, accuracy plateaus.

Subsampling Trap

Rare failures don't survive 1% sampling. Worst failures are always the rarest.

	LLM JUDGE	SLM JUDGE
Cost/1M evals	\$10k-50k	\$200-1k
Latency	1-3 sec	15-150ms
Real-time	Not feasible	Feasible
100% coverage	Prohibitive	Standard
Accuracy	90-95%	90-95%+

SLMS EXCEL AT

- Binary classification
- High-volume eval
- Real-time <100ms
- On-prem / private

SLMS STRUGGLE WITH

- Subjective quality
- Novel failure modes
- Evolving criteria
- Multi-factor trade-offs

SIZE	LATENCY (L4)	BEST FOR
3B	15-60ms	Real-time guardrails, cost-sensitive
8B	50-150ms	Async monitoring, strict >95%

LUNA (GALILEO)

<50ms

Purpose-built eval SLM

- Llama + Mistral (3B, 8B)
- Single-token output
- 1/50th cost of GPT-5
- Log-probs, not text gen
- LoRA multi-metric adapters
- Hallucination + adherence

Llama Guard (Meta)

8B beats GPT-4 on safety. 1B INT4: 440MB on phone. 1.3% F1 drop.

PHUDGE (Phi-3)

3.8B. SOTA 4 benchmarks. Classification beats generation.

Output Modes

MODE	LATENCY	USE CASE
Single-token	15-50ms	Logits. Real-time guardrails.
Verdict-only	50-100ms	PASS/FAIL. Routing.
Reasoning	200-500ms	Explanations. Audit trails.

TRAINING DATA

Test set (first): 300-500 SME-labeled, min 100/class

Training set: 1K-10K, balanced 50/50

Augmentation: Synthetic from LLMs, SME-verified

Hard negatives: Looks like violation, isn't

Document all: Source, labeler, date, version

The SLM insight: when you run GPT-4 as a toxicity judge, you're paying for its knowledge of Shakespeare and Baroque architecture. An SLM trained for toxicity dedicates its entire capacity to that single judgment.

Guardrails & Reference

Which Metrics (By Use Case)

RAG APPLICATION

RETRIEVAL	GENERATION
• Chunk Attribution >80% • Chunk Utilization >80% • Chunk Relevance >90%	• Context Adherence* >80% • Completeness >90% • Groundedness >90%

AGENT / MULTI-STEP

- Tool Selection Quality (1.0)
- Tool Error (1.0)
- Action Completion (binary)
- Agent Efficiency
- Agent Flow

Galileo: 8 agent evals out-of-box

GENERAL LLM

- | | |
|---|---|
| <ul style="list-style-type: none">• Instruction Adherence >0.9• Conciseness | <ul style="list-style-type: none">• Correctness* >90%• Tone (brand voice) |
|---|---|

Pain Points Solved

Infinite loops → Agent Efficiency detects repeated actions

Wrong tools → Tool Selection Quality catches mistakes

Late guardrails → Sub-150ms SLM blocks before delivery

Unreliable judges → ChainPoll voting reaches 95%+

Redeploy friction → Centralized prompt management

RESOURCES

- | | |
|-------------|---|
| Free Course | galileo.ai/evalengcourse |
| Docs | v2docs.galileo.ai |
| Luna-2 | galileo.ai/luna-2 |
| ChainPoll | arxiv.org/abs/2310.18344 |

Production Guardrail Actions

- **Block risky outputs.** Score below threshold = blocked before delivery
- **Route by confidence.** Low to human review, high auto-approves
- **Trigger HITL.** Uncertain cases escalated to experts
- **Enforce policy.** Real-time scores drive governance
- **Capture and refine.** 100% observability feeds back into eval loop

7 Mistakes to Avoid

- ✗ Too many metrics: start with one binary judge
- ✗ Generic off-the-shelf: tune for your domain
- ✗ Outsource error analysis: look at data yourself
- ✗ Likert 1-5: use binary Pass/Fail instead
- ✗ No few-shots: critical for calibration
- ✗ Same model judge + gen: 10-25% bias
- ✗ Single judge: ChainPoll voting (3/5/7)

Prompt Template (ChainPoll)

```
You are an expert evaluator for [DOMAIN].

EVALUATION CRITERIA:
1. [Criterion]: [Definition with examples]
2. [Criterion]: [Definition with examples]

SCORING:
"Pass": Meets all (minor issues OK)
"Fail": Significant errors/omissions

<examples> <example>
<input> <response> <critique> <judgment>
</example> </examples>

Evaluate step by step, then:
{"reasoning":"...", "judgment":"Pass/Fail"}
```

CHAPTER 06

Eval Engineering Checklist

Where are you? Check off what you've done. Focus on the next unchecked item.

PRE-PRODUCTION

Build the foundation

POST-PRODUCTION

Ship and monitor

AT SCALE

Optimize cost and coverage

 Define Quality

PRE-PRODUCTION

- ☐ Identify one domain expert as your "benevolent dictator"
- ☐ Write down what "good" means for your use case in specific terms
- ☐ Define 3-5 failure modes you care about most
- ☐ Create a diverse test dataset (200-500 examples from production)

 Build First Judge

PRE-PRODUCTION

- ☐ Start with code-based evals for deterministic checks
- ☐ Write your first LLM judge prompt with explicit criteria
- ☐ Add 5-10 few-shot examples (balanced pass/fail)
- ☐ Include edge cases that cause disagreement
- ☐ Use binary Pass/Fail, not Likert scales
- ☐ Require chain-of-thought reasoning before verdict

 Refine with SME

PRE-PRODUCTION

- ☐ Have expert label 100+ examples with reasoning
- ☐ Run disagreement analysis (find where judge and expert differ)
- ☐ Iterate prompt until > 90% agreement (F1 > 0.85)
- ☐ Split ground truth: Train 20% / Dev 40% / Test 40%
- ☐ Document criteria version and all edge case decisions

 Set Up ChainPoll

PRE-PRODUCTION

- ☐ Configure 5 parallel judges at temperature 0
- ☐ Validate AUROC against your ground truth
- ☐ Test for position, verbosity, and self-preference bias

 CI/CD Integration

POST-PRODUCTION

- ☐ Add eval suite to your CI/CD pipeline
- ☐ Block deploys that fail quality thresholds
- ☐ A/B test every prompt change against scenario bank
- ☐ Set up regression testing for known failure modes
- ☐ Track latency, token usage, cost per task as baselines

 Production Monitoring

POST-PRODUCTION

- ☐ Enable observability on 100% of production traffic
- ☐ Set up alerts for eval score drops below threshold
- ☐ Configure confidence-based routing (low to human, high to auto)
- ☐ Implement real-time guardrails for safety and compliance
- ☐ Log all evaluation decisions for audit trails

 Continuous Improvement

POST-PRODUCTION

- ☐ Review production failures weekly
- ☐ Add new failure modes to ground truth dataset
- ☐ Refresh ground truth quarterly
- ☐ Re-run disagreement analysis after criteria changes
- ☐ Track F1 trend over time (should only go up)

 Team Operations

POST-PRODUCTION

- ☐ Document all eval criteria in a shared rubric
- ☐ New hires onboard by reading test cases
- ☐ Product team references specific failing cases, not vibes

 SLM Transition

AT SCALE

- ☐ Create 300-500 SME-labeled test set (min 100/class)
- ☐ Build training set: 1K-10K balanced examples
- ☐ Generate synthetic data, verify with SMEs before use
- ☐ Include hard negatives in training data
- ☐ Select model size: 3B (real-time) or 8B (accuracy)
- ☐ Fine-tune with LoRA for multi-metric evaluation

 Production SLM Deployment

AT SCALE

- ☐ Validate SLM accuracy matches LLM judge on test set
- ☐ Deploy single-token mode for real-time guardrails
- ☐ Move from sampling to 100% traffic evaluation
- ☐ Set up confidence routing with SLM scores
- ☐ Monitor for distribution drift between training and production

 Cost and Governance

AT SCALE

- ☐ Track cost reduction vs LLM judge baseline
- ☐ Measure latency: target sub-150ms for guardrails
- ☐ Set up policy enforcement using real-time eval scores
- ☐ Configure human-in-the-loop for uncertain cases
- ☐ Build feedback loop: production failures feed back into training

“**Eval engineering isn't testing.**
It's building **reusable Intellectual Property**
that governs your AI system in production.”
Ship reliable AI.

